Memory-safety & concurrency defect classes

All are silent in C++: accepted by the compiler, surfacing only later as corruption, a crash, a hang, or a latency spike

Defects in C++ control-plane code
(each has a natural incarnation in UECM)

Memory-safety family

object lifetime and bounds

- **Use-after-free ★**
- **Double-free ★**
- **Use-after-move ★**
- Buffer overflow (not injected)

Concurrency family

unsynchronised or ill-ordered access

- **Data race ★**
- **Iterator invalidation ★**
- **Unguarded shared mutation ★**
- **Thread-unsafe handle shared ★**

Boundary group

chosen because safe Rust does not prevent them

- **Lock-order deadlock ◆**
- **Lost update ◆**
- **Reference-cycle leak ◆**
- **Integer overflow (release) ◆**
- **Worker-failure blast radius ◆**
- **Double-free across the seam ◆**
- **Forgotten lock guard ◆**

Legend

- ★ injected, inside Rust's guarantee (7)
- ◆ injected, boundary of the guarantee (7)
- described for completeness, not injected

Both sides are the same size on purpose. A catalogue holding only the defects Rust prevents would confirm its own premise, so this one measures a boundary instead.